Applied Machine Learning for Business Analytics

Lecture 7: Building LLM Application: Prompt Chaining & RAG

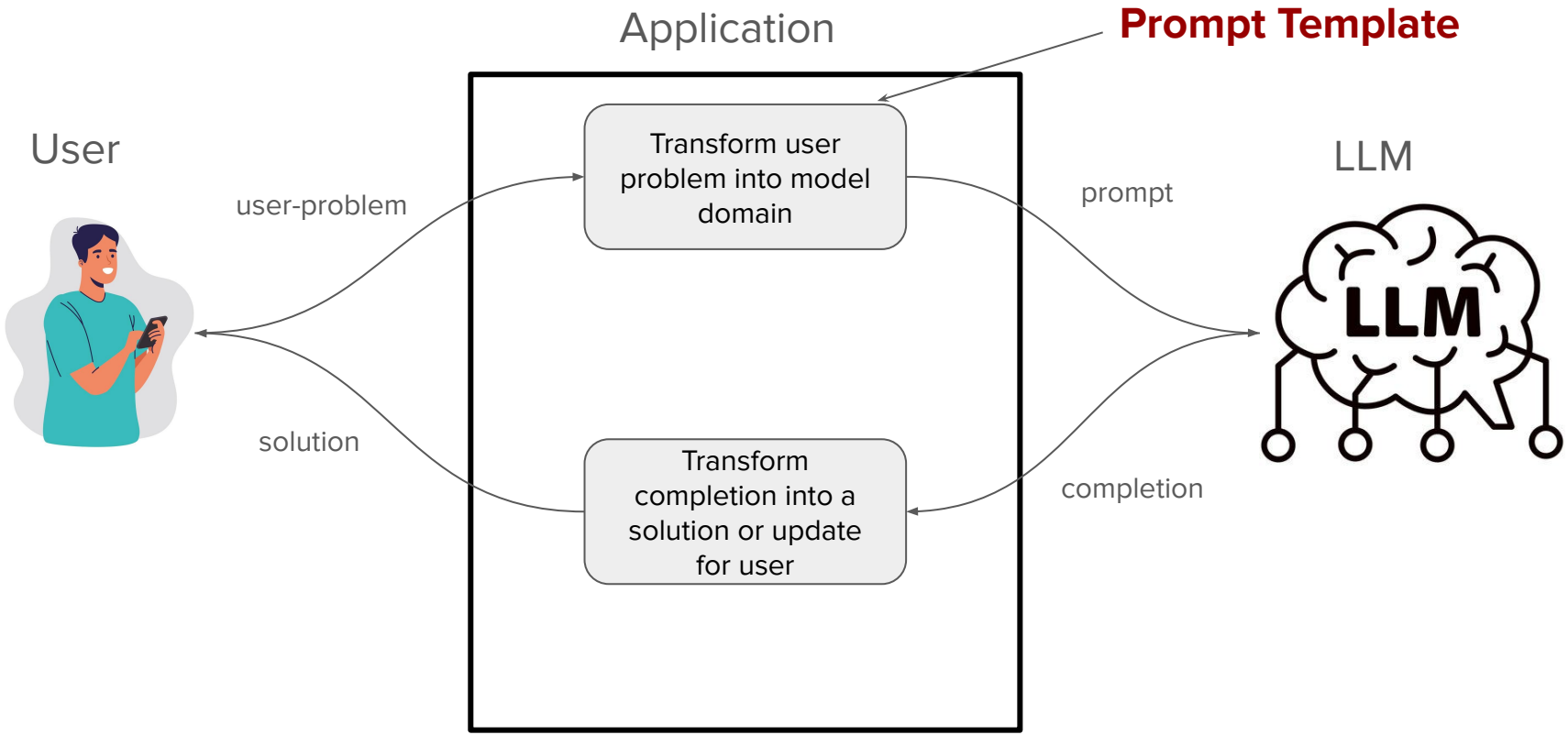
Lecturer: Zhao Rui

1. LLM Applications

What is an LLM Application

- A layer that makes the raw LLM usable for real tasks
  - System prompt
  - Input formatting & preprocessing
  - Output parsing & post-processing
  - User interface (chat, API, form)
  - Basic memory (conversation history)
- Common Examples
  - Email generator
  - RAG
  - Summarizer
  - Customer support chatbot
  - etc

Building LLM Applications



2. Prompt Chaining

Chain complex prompts

Complex Prompt (single step)

Prompt:

"Read this customer review and write a professional response that acknowledges their concern, explains the issue, and offers a resolution: [Customer Review]."

Customer Review:

"I ordered a laptop, but it arrived three days late. The packaging was also damaged. This was very disappointing since I needed it urgently for work."

Output:

"Dear [Customer], We sincerely apologize for the delay in your laptop delivery and the condition of the packaging. This is not the experience we want for our customers. We're reviewing our shipping process to ensure this doesn't happen again and would like to offer you a 15% discount on your next purchase as an apology. Thank you for bringing this to our attention."

Prompt 1: Extract Key Issues

"Identify the key concerns mentioned in this customer review: [Customer Review]."

Output:

- Delivery Delay: Laptop arrived three days late.
- Packaging Issue: Packaging was damaged.
- Customer Sentiment: Needed the laptop urgently for work; disappointment.

Prompt 2: Create a Response Structure

"Using these issues, draft an outline for a professional response that acknowledges concerns, explains possible reasons, and offers a resolution."

Output:

1. Acknowledge the delay and damaged packaging.
2. Explain shipping delays (e.g., logistics or warehouse issues).
3. Apologize for the inconvenience and offer a resolution (e.g., discount or expedited future service).

Prompt 3: Write the Full Response

"Using this outline, write a professional response: [outline]."

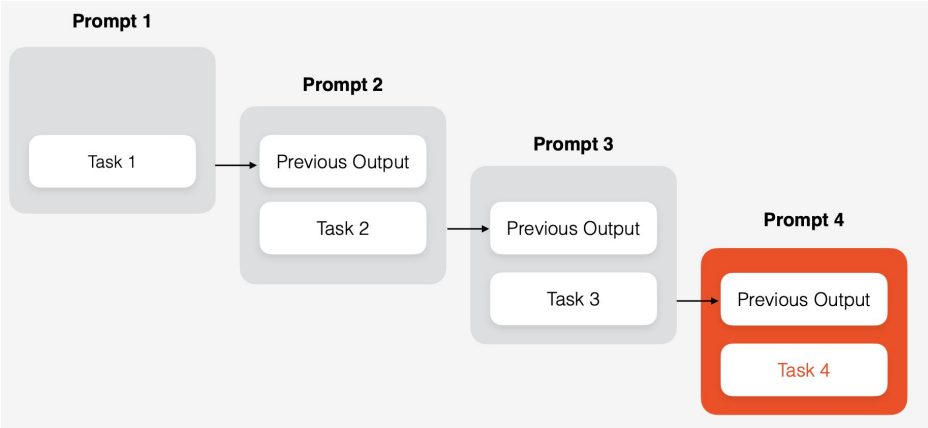
Output:

"Dear [Customer], We sincerely apologize for the delay in your laptop delivery and for the damaged packaging. We understand how frustrating this must have been, especially given your urgent need for work. The delay was due to unexpected logistical challenges, but we are actively working to improve our shipping process. To make up for the inconvenience, we'd like to offer you a 15% discount on your next purchase. Please let us know if there's anything more we can do for you."

Source: https://docs.anthropic.com/en/prompt-library/library

Prompt Chaining

- LLM perform better when give one focused task at a time. Prompt chaining splits a complex request into smaller steps - the output of each stem becomes the input of the next.
  - The chain is predefined while LLM is only executing.
  - No autonomy



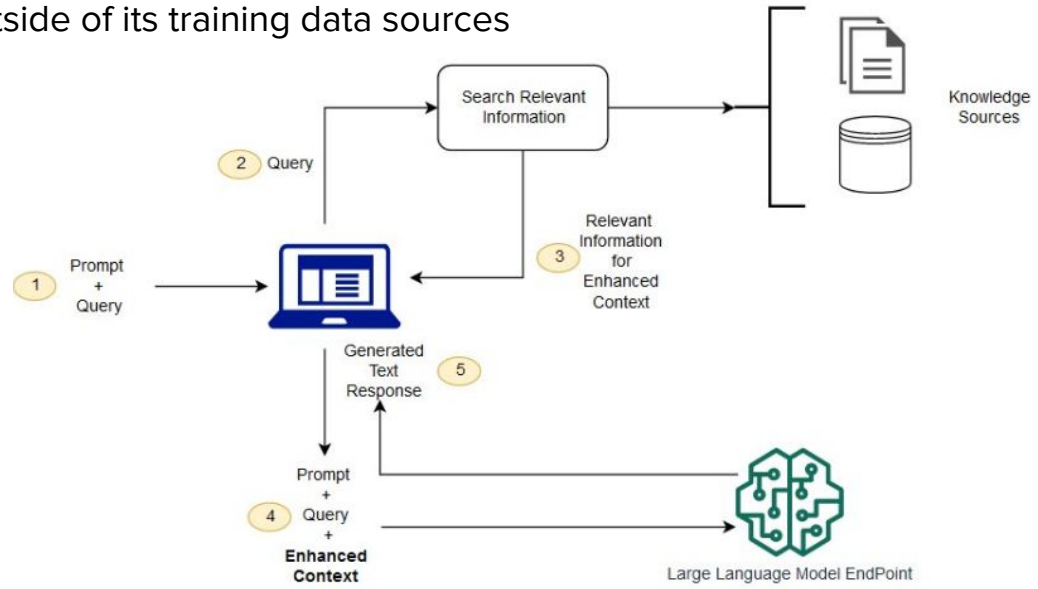
Single Prompt vs Prompt Chaining

Single Prompt	Prompt Chaining
LLM does everything at one shot	One clear task per step
One mistake ruins everything	Error caught step by step
Black box output	Each step is inspectable

3. RAG

What is RAG

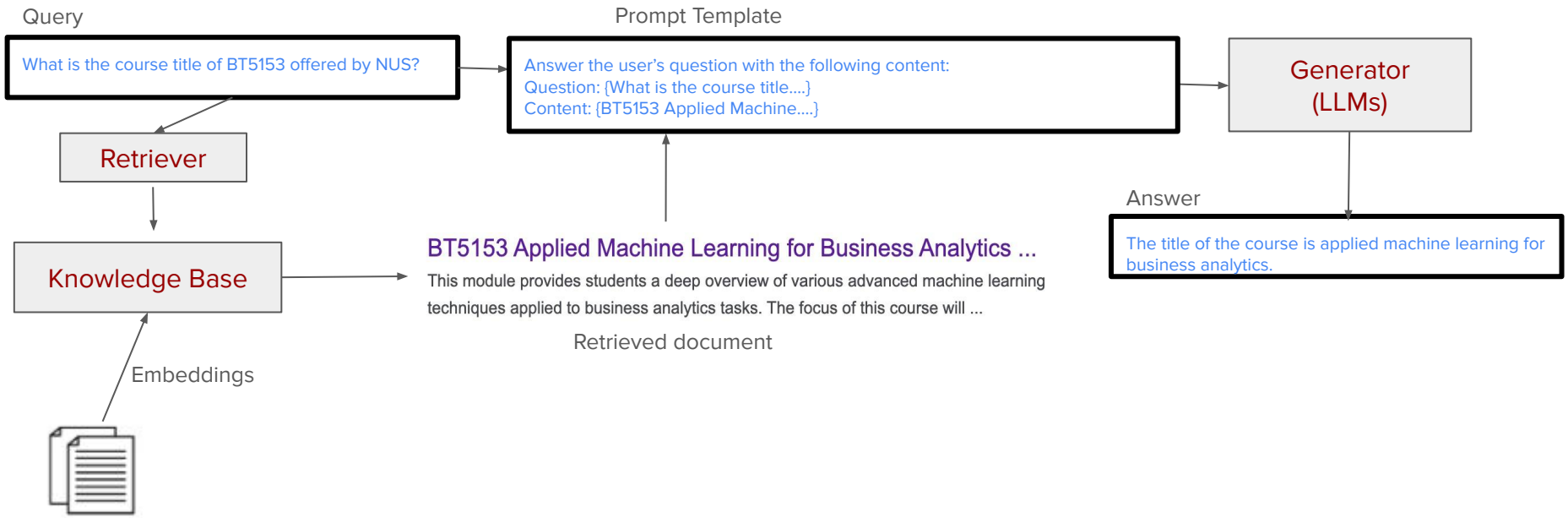
- Retrieval-Augmented Generation
  - A process to optimize the output of a LLM, so it references an authoritative knowledge base outside of its training data sources before responding.
- Benefits of RAG
  - Cost-effective implementation
  - Current information
  - Enhanced user trust
  - More develop control



source: https://aws.amazon.com/what-is/retrieval-augmented-generation/

One trivial example

- LLM is trained before the course website was built



It can be in webpage, pdf files, or even images/videos.

Old wine in new bottles

- RAG is not:
  - A new idea
  - A framework
- RAG is just combining Retrieval and Generation
  - Retrieval comes from Information Retrieval (IR):
    - The process of obtaining relevant information based on a user' information need expressed as a query/question
    - It is a research topic in computer science for decades
  - Generation is handled by LLMs

“Good” RAG

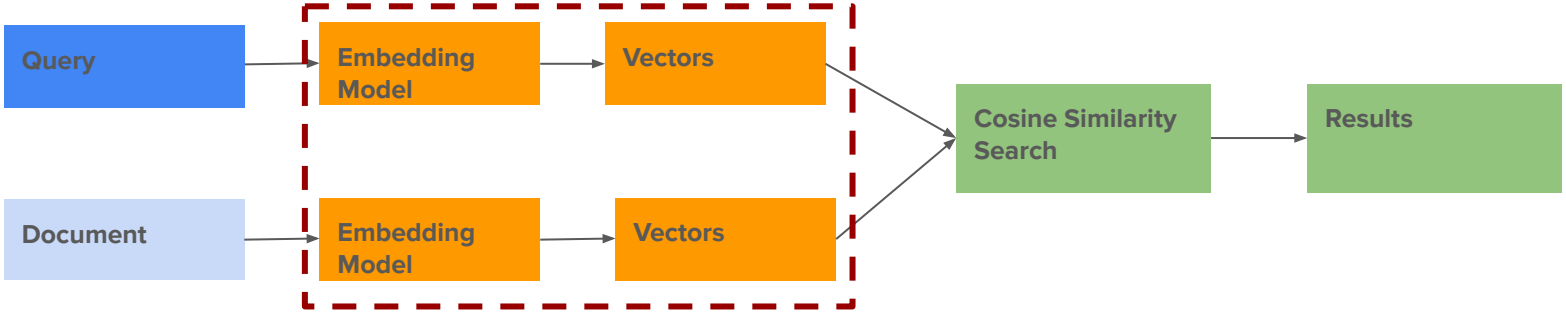
- **Good retrieval pipeline**
- Good generative model
- Good way of linking them up

Performance issues with Naive RAG

- Bad Retrieval
  - Low precision: not all retrieved chunks are relevant
    - Hallucination + Wrong replies
  - Low recall: not all relevant chunks are retrieved
    - Lacks enough context for LLM to synthesize the answer
  - Outdated information: the knowledge base is out of date
- Bad Response Generation (Native LLM Issues)
  - Hallucination: Model makes up an answer that isn’t in the context
  - Irrelevance: Model makes up an answer that does not answer the question
  - Toxicity/Bias: Model makes up an answer that is offensive

Start from Basic - Retrieval Pipeline

This is called a Bi-encoder approach



Start from Basic - Retrieval Pipeline

```
[ ] from sentence_transformers import SentenceTransformer
    model = SentenceTransformer('paraphrase-MiniLM-L6-v2')
```

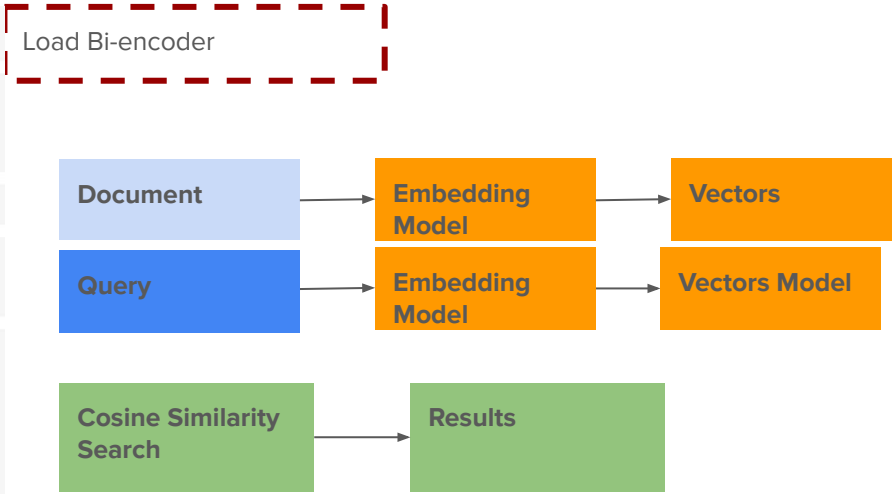
```
[ ] from wikipediaapi import Wikipedia
    wiki = Wikipedia('RAGBot/0.0', 'en')
    doc = wiki.page('Rock_Lee').text
    paragraphs = doc.split('\n\n')
```

```
[ ] docs_embed= model.encode(paragraphs,normalize_embeddings=True)
```

```
[ ] # Embedt h equery
    query = "How did Rock Lee open his chakra gates?"
    query_embed= model.encode(query,normalize_embeddings=True)
```

```
[ ] import numpy as np
    import torch

    similarities= np.dot(docs_embed,query_embed.T)
    # Convert the NumPy array to a PyTorch tensor
    similarities_tensor = torch.tensor(similarities)
    # Now apply topk to the tensor
    top_3_idx= similarities_tensor.topk(3).indices.tolist()
    most_similar_documents= [paragraphs[idx] for idx in top_3_idx]
```



Vector DB

- As the crucial part of RAG pipeline, search index supports the retrieval of content based on query
  - Indexing
    - Naive implementation: flat index
      - A brute force distance calculation between the query vector and all the chunks' vectors
  - Retrieving (nearest neighbours searching)
    - Efficient framework:
      - Open-source libraries: Faiss, Annoy
      - Managed solutions: Pinecone

Numpy is the vector DB

- The vector DB (or an index)
  - Allow Approximate search to avoid computing too many distance scores
  - It is only applicable for large scales retrieval
- Any modern CPU can search through hundreds of vectors in milliseconds

Dense Representation

- Vectorisation
  - The embedding model should be selected.
  - Search optimized models:
    - The leaderboard for massive text embedding benchmark

Rank	Model	Model Size (GB)	Embedding Dimensions	Sequence Length	Average (56 datasets)	Classification Average (12 datasets)	Clustering Average (11 datasets)	Pair Classification Average (3 datasets)	Reranking Average (4 datasets)	Ret Ave (15 dat
1	UAE-Large-V1	1.34	1024	512	64.64	75.58	46.73	87.25	59.88	54.
2	voyage-lite-01-instruct		1024	4096	64.49	74.79	47.4	86.57	59.74	55.
3	Cohere-embed-english-v3.0		1024	512	64.47	76.49	47.43	85.84	58.01	55

Dense Representation

- Compressing information from tokens to a single vector is bound to lose information
- Embeddings learned is limited to the training data of the embeddings models
- Fixed Vocabulary
  - Llama 3 2024 => [‘ll’, ‘##ama’, ‘3’, ‘202’, ‘##4’]

Chunk sizes

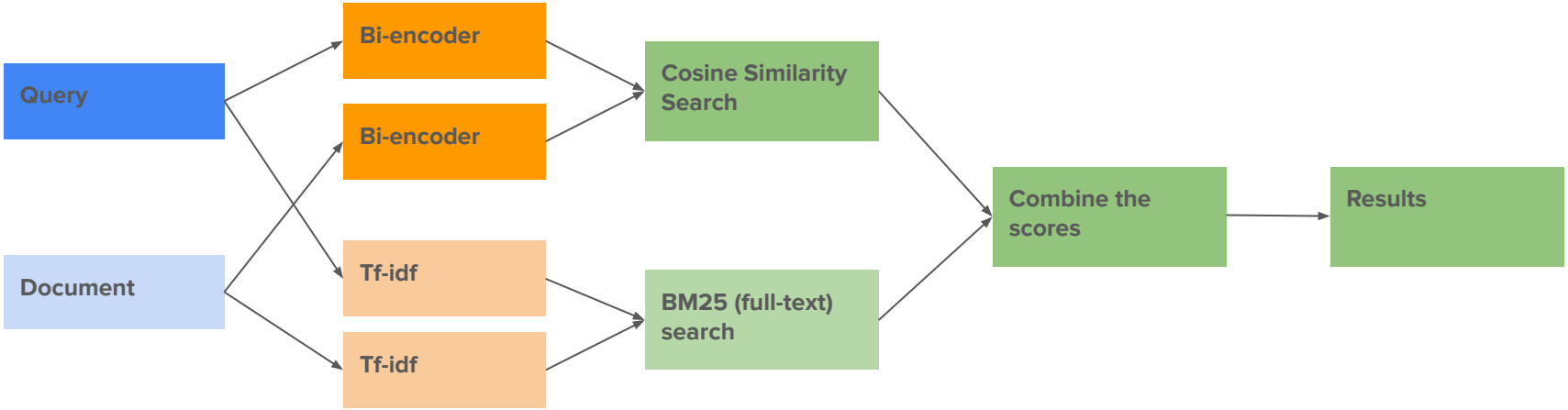
- Chunking
  - Split texts into chunks of some size without losing their meaning
  - The size of the chunk is also a hyper-parameter to be tuned
    - Embeddings models' capacity
    - Conflicts between enough context for LLM to reason (wide) and specific enough text embedding (narrow) in order to efficiently execute search upon
  - A few chunking techniques:
    - Fixed-size chunking
    - Content-aware chunking
      - Sentence splitting: sentence-level chunking
      - Specialized chunking: preserve the original structure of the content if it is in markdown, latex or other formats
    - A good survey here.

Sparse Representation - BM25

- Keyword search, is built on an old technology: BM25
  - Based on TF-IDF (sparse)
  - Pretty powerful on longer documents and documents containing a lot of domain-specific jargon
  - It’s inference-time compute overhead is quite tiny
- Free lunch for any pipeline

Hybrid: dense + sparse

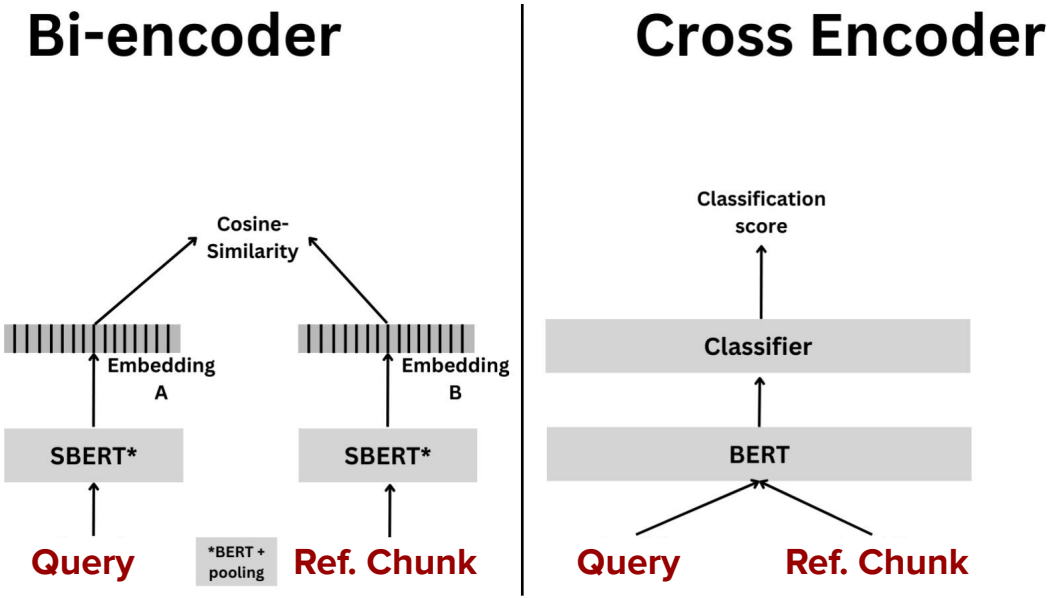
- Combine spread and dense representations



Bi-encoder

- Bi-encoders are (generally) used to create single-vector representations. They pre-compute document representations.
- Documents and query representations are computed entirely separately, they are not aware of each other
- Thus, document vectors can be pre-computed and at inference, encode your query and search for similar vectors
  - Very computationally efficient
  - It comes with retrieval performance tradeoffs.

Bi-encoder vs Cross Encoder



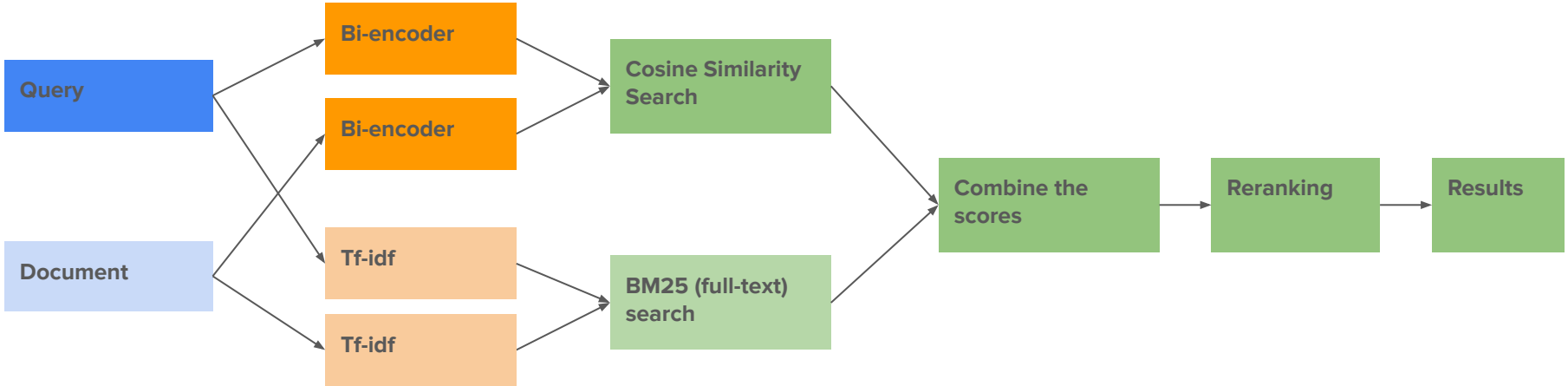
Source: https://osanseviero.github.io/hackerllama/blog/posts/sentence_embeddings2/

Given 10 queries and 1000 chunks in the reference:

- For Bi-encoder, how many times that the transformer would be called?
- For Cross-Encoder, how many times that the transformer would be called?

Reranker: The power of Cross-Encoders

- Leverage a powerful but computationally expensive model to score only a subset of your documents, previously retrieved by a more efficient model.
- One approach: cross-encoder
  - It is effectively a binary classifier
    - The input is a pair of query and document
    - The output is the probability of being the positive class as the similarity score



Metadata filter

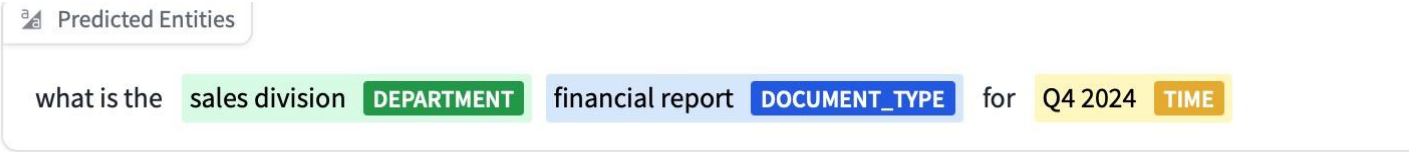
- Documents do not exist in a vacuum. There is a lot of metadata around them, some of which can be very informative
- Metadata is the context you can add with each text chunk
  - Examples
    - Page number
    - Document title
    - Summary of adjunct chunks
    - Hypothesis questions (reverse HyDE)
      - Ask the LLM to generate a question for each chunk
  - Benefits
    - Can help retrieval
    - Can augment response quality
    - Integrates with Vector DB metadata filters

Metadata filter

- Query: what is the sales division financial report for Q4 2024?
- Raw top-k retrieval via embeddings/keyword would have low precision
  - The model should accurately capture all of the key meaning: financial report, sales division, Q4 and 2024.
  - And the k also can not be set too high. Otherwise irrelevant financial reports would be passed to LLM

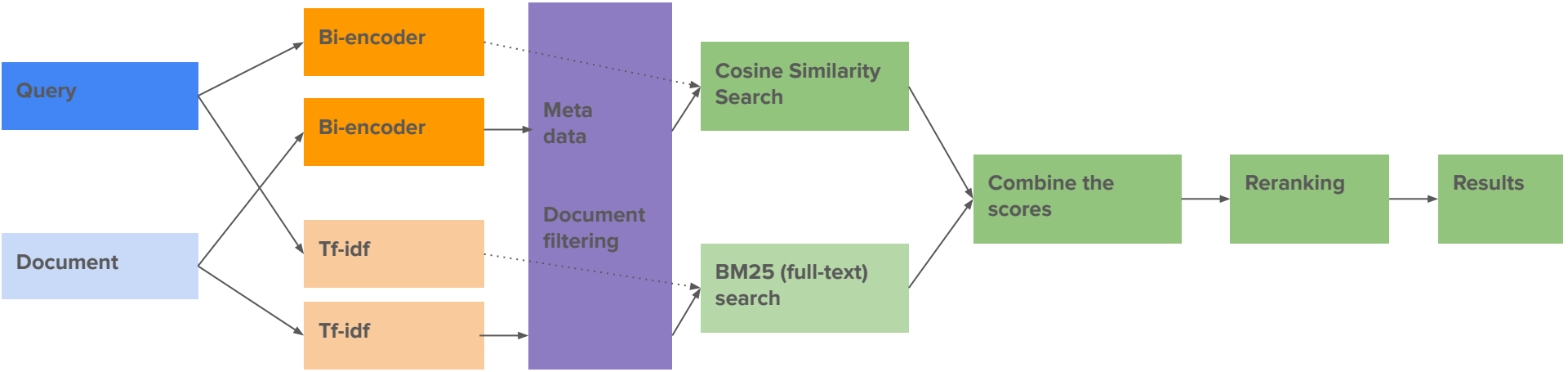
Metadata filter

- Apply entity detection models on query such as GliNER
- Ensure the business/query-relevant information is stored alongside their associated documents
- Than, the extracted entities could be used to pre-filter the documents, ensuring we only perform search on documents whose attributes are related to the query.



source: https://huggingface.co/spaces/urchade/gliner_mediumv2.1

The Final MVP



The Final MVP

```
[ ] from wikipediaapi import Wikipedia
    wiki = Wikipedia('RAGBot/0.0', 'en')
    docs = [{'text': x, 'category': "anime"} for x in wiki.page("Rock_Lee").text.split('\n\n')]
    docs += [{'text': x, 'category': "sports"} for x in wiki.page("Rock_Lee_(basketball)").text.split('\n\n')]
```

```
[ ] import lancedb
    from lancedb.pydantic import LanceModel, Vector
    from lancedb.embeddings import get_registry
    from lancedb.rerankers import CrossEncoderReranker
```

```
[ ] model = get_registry().get("sentence-transformers").create(name="paraphrase-MiniLM-L6-v2")
```

Bi-encoder

```
[ ] class Document(LanceModel):
      text: str = model.SourceField()
      vector: Vector(384) = model.VectorField()
      category: str
    db = lancedb.connect('.my_db')
    tbl = db.create_table("my_docv1", schema=Document)
```

Meta data

```
[ ] tbl.add(docs)
    tbl.create_fts_index("text")
```

BM25 (full-text) search

```
[ ] reranker = CrossEncoderReranker()
    query = "How did Rock Lee open his chakra gates?"

    # Apply filter within the search query
    results = (tbl.search(query, query_type='hybrid').where("category='anime'").limit(3).rerank(reranker=reranker))
```

Document Filtering

Demo

```
all_data = results.to_list()[:3]
[ele['text'] for ele in all_data[:3]] # get top3 results
```

```
['Appearances\nIn Naruto\nRock Lee is a ninja from Konohagakure part of Team Guy, a four-man cell of ninja led by Might Guy.
Inspired by Lee\'s determination to become stronger despite his inability to perform basic ninja techniques, Guy takes a
personal interest in him, deciding to help him achieve his dream of becoming a powerful ninja by using only taijutsu that is
primary hand-to-hand combat. This relationship with Guy causes Lee to acquire many of Guy\'s traits. Lee believes he can
surpass the natural talents of others through hard work and passion; throughout the series, he attempts to surpass Neji Hyuga,
who is labeled a "genius". Lee first appears in the series as a participant in the Chunin Exams, twice a year exams for ninja
who wish to increase their rank. During the Chunin Exams, Lee battles Gaara, a ninja from the village of Sunagakure. In the
fight, Lee opens the five of the eight chakra gates, limits on the body\'s ability to use chakra, using a forbidden technique
known as the Hidden Lotus, increasing his natural abilities at the cost of his health. Despite his effort, Gaara cripples Lee
by crushing his left arm and leg, injuring Lee to the point that he must abandon being a ninja.\nWhen Tsunade, a Konohagakure
medical ninja, returns to lead the village as the Fifth Hokage, she offers to operate on him. Despite the procedure\'s fifty
percent chance of failure, Guy encourages Lee to have the operation. Ultimately, Lee undergoes the surgery, which succeeds in
healing his arm and leg. Immediately after the operation, Lee follows a team of ninja led by Shikamaru Nara who attempt to stop
Sasuke Uchiha from defecting from Konohagakure to the village of Otogakure. Lee battles the Otogakure ninja Kimimaro through
using the Potion Punch (酔拳, Suiken, literally "Drunken Fist", English TV: "Loopy Fist") fighting style in which he becomes
inebriated with unpredictable attacks. When Kimimaro is on the verge of defeating Lee, Gaara intervenes, continuing the
battle.\nIn Part II, Lee obtains the rank of Chunin, and is dispatched with his team to help save Gaara following his abduction
by the criminal organization Akatsuki. During the events of the Fourth Shinobi War Lee is assigned to the Third Division, Lee
helps in fighting the Kabuto Yakushi\'s reanimated army and later aids Naruto Uzumaki in the fight against Obito Uchiha and
Madara Uchiha. Years after the war, Lee marries an unknown woman and has a son named Metal Lee. In the epilogue, Lee is last
seen many years later, training with his son. In Boruto: Naruto the Movie, Lee hosts the third stage of the Chunin Exam.',
 'Reception\nLee has ranked highly in the Shōnen Jump popularity polls for the series, initially continuously placing in the
top ten and reaching fifth place once. In later polls, Lee lost his top ten status. Also, several pieces of merchandise based
on Lee have also been released, including action figures of his Part I and Part II appearances, plush dolls, and keychains. My
Hero Academia author Kōhei Horikoshi said that Rock Lee\'s inhability to use ninjutsu and instead use hand-to-hand combat made
him highly stand out in the manga. He also praised how heroic is Rock Lee\'s portrayal in the Chunnin Exams when facing
Orochimaru\'s underlings.\nSeveral publications for manga, anime, video games, and other media have provided commentary on
Lee\'s character. IGN\'s A.E. Sparrow called Lee one of his favorite characters in the series and compared his personality to
that of Bruce Lee and Noel Gallagher. Fellow editor Ramsey Isler ranked him as the eight best character on the series and said
he "was the true underdog of the series." Isler added, "Perhaps a little too intense, but always fiercely devoted to his cause,
Rock Lee added all sorts of flavor to the series." However, Rock Lee\'s profile on IGN describes him as "kind of stiff" because
of his very polite demeanor. Active Anime celebrated Lee\'s introduction in the series as a comedic relief to the growing
tension of the story at that point. Anime Insider listed him in their top five list for "pure-hearted heroes" from anime and
manga publications, ranking at number five. Insider praised him for "never [giving] up, even in the face of people with actual
ninja powers."\nAnime News Network referred to Lee as the "star of [the Chunin Exam arc]", and claimed that he "almost single-
handedly rescues this arc from being tossed into the \'entertaining but disposable\' bin". His fight against Gaara in the exams
was listed as second best one in anime by AnimeCentral. Anime News Network also called Lee the "goofiest looking character" in
the series and praised Kishimoto\'s "ninja-punk visual sensibilities" that allowed him to make Lee "damn cool when the action
starts". In the NEO Awards 2007 from Neo, Rock Lee won in the category "Best Anime Character". He was also listed as one of the
three "Honorable Mentions" from Naruto by Wizard Entertainment\'s Danica Davidson with comments from the article being focused
on Lee\'s determination. When the Naruto manga ended, writer Yūto Kubota expressed that Lee was his favorite character.',
 'Rock Lee (Japanese: ロック・リー, Hepburn: Rokku Rī) is a fictional character in the anime and manga series Naruto and Naruto
Shippuden created by Masashi Kishimoto. At first Masashi designed Lee to symbolize human strength. In the anime and manga, Lee
is a ninja affiliated with the village of Konohagakure, and is a member of Team Guy, which consists of himself, Neji Hyuga,
Tenten, and Might Guy—the team\'s leader. Unable to use most ninja techniques, Lee dedicates himself  to using solely taijutsu,
ninja techniques similar to martial arts. Lee dreams of becoming a "splendid ninja" despite his inabilities. Lee has appeared
in several pieces of Naruto media, including the third and fourth featured films in the series, the third original video
animation, and multiple video games.\nNumerous anime and manga publications have commented on Lee\'s character. IGN compared
Lee to Bruce Lee and Noel Gallagher, and Anime News Network called Lee the "goofiest looking character" in the series. Among
the Naruto reader base, Lee has been popular, placing high in several popularity polls. Numerous pieces of merchandise have
been released in Lee\'s likeness, including figurines and plush dolls.']
```

Code:
https://colab.research.google.com/drive/1L0VWAgliywqjNE1TFmdfusAV86yDuR94?usp=sharing

How to Improve RAG

- Data
  - Store additional information beyond raw text chunks
- Embeddings
  - Optimize embedding representations
- Retrieval
  - Advanced retrieval instead of top-k embedding lookup
- Synthesis
  - LLMs can play a big role in the process

How do we properly evaluate a RAG

- After those changes, we need to evaluate the impacts
  - Evaluate various components
    - Is the retrieval good?
    - Is the LLM good?
  - Evaluate end-to-end

“Advanced” Topics for RAG

- Chunking Size
  - Divide large documents into small chunks
- Query Transformation
- Parametric Retrieval Augmented Generation

See more in the appendix

Parametric Retrieval Augmented Generation

Weihang Su
swh22@mails.tsinghua.edu.cn
DCST, Tsinghua University
Beijing 100084, China

Yichen Tang*
DCST, Tsinghua University
Beijing 100084, China

Qingyao Ai†
aiqy@tsinghua.edu.cn
DCST, Tsinghua University
Beijing 100084, China

Junxi Yan
DCST, Tsinghua University
Beijing 100084, China

Changyue Wang
DCST, Tsinghua University
Beijing 100084, China

Hongning Wang
DCST, Tsinghua University
Beijing 100084, China

Ziyi Ye
DCST, Tsinghua University
Beijing 100084, China

Yujia Zhou
DCST, Tsinghua University
Beijing 100084, China

Yiqun Liu
DCST, Tsinghua University
Beijing 100084, China

4. Use LangChain to Build RAG

Build RAG quickly

```python
print("Creating RAG chain...")
# Initialize LLM (OpenAI)
llm = ChatOpenAI(temperature=0)
# Create prompt template
template = """Use the following pieces of context to answer the question. If you don't know the answer, just say that you don't know.

Context: {context}
Question: {question}

Answer:"""
prompt = PromptTemplate.from_template(template)
# Format documents function
def format_docs(docs):
    return "\n\n".join(
        f"{doc.page_content}\n(Source: {doc.metadata['source']}, Page: {doc.metadata['page']})"
        for doc in docs
    )
# Create the RAG chain
# RunnablePassthrough() is used to pass the question through the chain unchanged. It means that the question firstly pass through the retriever
# then format the context. And than the question would be passed with the retrieved context to the prompt
rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)
# StrOutputParser() is used to parse the output of the llm to a string.
# Here, it is the ChatModel. So it extracts the .content attribute of the message, ensuring that the final output is in string format.
```

Test the flow

- Given the question: How is the dataset prepared to detect the AI-generated essays
- Knowledge base: Project paper from last year
- Two LLMs are checked:
  - GPT-40
  - Qwen2.5-0.5B

Detection of AI-Generated Text for Essay Competitions

Abstract

The widespread adoption of large language models like GPT (Generative Pre-trained Transformer) has facilitated the generation of text that closely mimics human writing, raising concerns about plagiarism and academic integrity. This project aims to develop a specialized system capable of detecting AI-generated text, particularly in the context of essay submissions to academic competitions. By leveraging advanced natural language processing techniques and pre-trained machine learning models, including XLNet, GPT-2.0, and DistilBERT, we conduct a comprehensive exploration encompassing two phases of fine-tuning. The initial phrase optimizes each model's performance with the most appropriate datasets and features whereas the subsequent phase refines the models within the specific domain of academic essays. Through meticulous evaluation and comparison, we select the most suitable model for integration into our detection system. By combining cutting-edge techniques with rigorous experimentation, we seek to enhance the credibility and fairness of academic assessments while promoting a culture of originality and integrity in scholarly work.

task by employing traditional classification algorithms, such as TF-IDF combined with a logistic regression model, and other traditional algorithms like support vector machine and random forest models. This approach was advantageous for its interpretability. Subsequently, the researchers explored deep learning approaches, specifically leveraging language models like pretrained BERT models. These models were fine-tuned using a meticulously curated dataset consisting of generated-text pairs. Notably, Rodriguez et al. demonstrated that even in situations with limited available resources, robust performance could be achieved by utilizing a few hundred labeled authentic and synthetic texts specific to the domain, without the need for complete information about the LLM text generation pipeline. Therefore, our proposed project adopted a similar methodology, aligning with the flow of Rodriguez et al.'s research.

Moreover, while studies have explored LLM-generated text detection, there is a need for specialized detectors tailored to the essay competition domain, incorporating domain-specific features and writing patterns. Efforts will

5. Basic Memory

LLMs are stateless

- LLMs do not have inherent memory
  - They treat each input **independently** and do not remember previous interactions

```python
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    ("human", "{input}")
])
```

✓ 0.0s

```python
llm = ChatOpenAI(model='gpt-4o')
chain = prompt | llm
result = chain.invoke({"input": "Hi, my name is Rui."})
print(result.content)
```

✓ 0.7s

```
Hello, Rui! How can I assist you today?
```

```python
result = chain.invoke({"input": "What is my name?"})
print(result.content)
```

✓ 1.8s

```
I'm sorry, but I don't have access to personal data about you, including your name. If you tell me your name, I can remember it for the duration of our conversation, but I don't have any way to know it otherwise.
```

ChatGPT 4o

Hi, this is Rui.

Hi Rui! What's on your mind today?

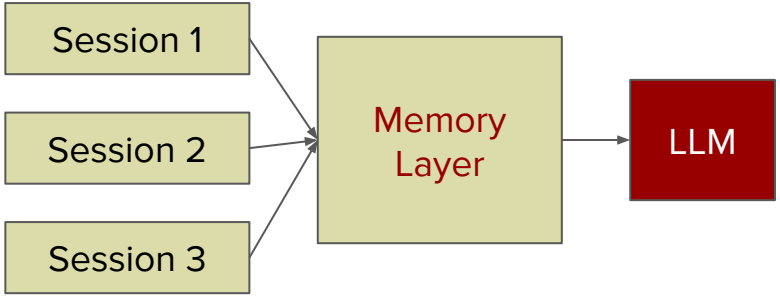
What is my name?

Your name is Rui. What can I do for you today, Rui?

Why the name can be remembered here?

LLM application, not LLM, manage memory

- LLM are stateless by design
  - Each API call starts with a blank status - no native memory
- Without memory, LLM applications fail at real tasks
  - Can not recall preferences, names or past decisions
  - Can not learn from mistakes across sessions
- Memory (a.k.a, context engineering)
  - The context window is limited
  - Choosing what to keep & how to compress those is the key challenge
  - Best LLM applications win on context quality, not just model quality



Memory Types

- **Working Memory : context windows**
  - Chat history within the current session
- Procedural Memory: system prompt rules
  - System prompt instructions that evolve with feedback
    - *LLM rewrites its own system prompt after reflection*
- Episodic Memory: past interactions
  - Past interaction summaries & few-shot examples
    - *Last time users asked about X, approach Y worked*
- Semantic Memory: facts & users profile
  - Users preferences, domain knowledge, structured facts
    - *Users preferes concise answers in English*

Conversation History as Memory

- The simplest memory technique:
  - Include past messages in every prompt
  - Memory: An ordered list of message objects: (role, content) with every new user input:
  - The model uses this history to understand the context of the current conversation and generate a relevant response

S: You are a helpful assistant
U: My name is RZ. I am at BT5153 class
A: Hello, RZ! How can I assist you today with your BT5153 class.
U: What is namy name? and what class am I in?
A: Your name is RZ, and you are in the BT51513 class. How can I assist you further?

```python
from openai import OpenAI
from dotenv import load_dotenv
import os
load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
# This list stores the conversation history
conversation_history = [
    {"role": "system", "content": "You are a helpful assistant."},
]
def get_chat_response(user_input):
    # Add the new user message to the history
    conversation_history.append({"role": "user", "content": user_input})

    # Send the entire history to the API
    response = client.chat.completions.create(
        model="gpt-3.5-turbo", messages=conversation_history
    )
    ai_response_content = response.choices[0].message.content
    # Add the AI's response to the history for the next turn
    conversation_history.append({"role": "assistant", "content": ai_response_content})
    return ai_response_content
# Example usage:
print(get_chat_response("My name is RZ. I am at BT5153 class"))
print(get_chat_response("What is my name? and what class am I in?"))
```

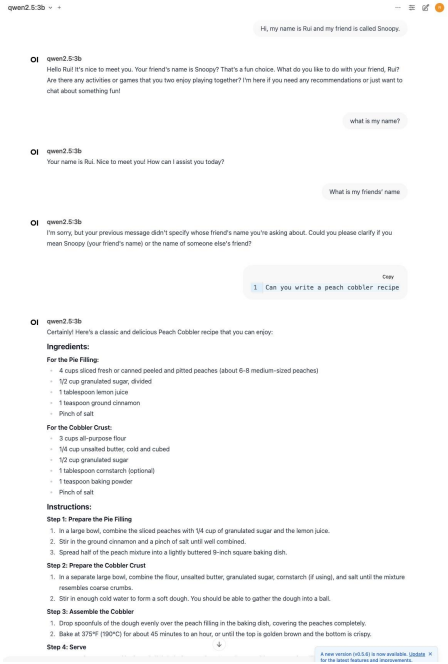
Basic Memory: OpenAI API vs LangChain

```python
from openai import OpenAI
from dotenv import load_dotenv
import os
load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
# This list stores the conversation history
conversation_history = [
    {"role": "system", "content": "You are a helpful assistant."},
]
def get_chat_response(user_input):
    # Add the new user message to the history
    conversation_history.append({"role": "user", "content": user_input})

    # Send the entire history to the API
    response = client.chat.completions.create(
        model="gpt-3.5-turbo", messages=conversation_history
    )
    ai_response_content = response.choices[0].message.content
    # Add the AI's response to the history for the next turn
    conversation_history.append({"role": "assistant", "content": ai_response_content})
    return ai_response_content
# Example usage:
print(get_chat_response("My name is RZ. I am at BT5153 class"))
print(get_chat_response("What is my name? and what class am I in?"))
```

```python
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import HumanMessage, AIMessage
from dotenv import load_dotenv
load_dotenv()
prompt = ChatPromptTemplate.from_messages(
    [
        ("system", "You are a helpful assistant."),
        MessagesPlaceholder(variable_name="history"),
        ("human", "{input}"),
    ]
)
llm = ChatOpenAI(model="gpt-3.5-turbo")
chain = prompt | llm
history = []
def chat(user_input: str):
    result = chain.invoke({"input": user_input, "history": history})
    history.append(HumanMessage(content=user_input))
    history.append(AIMessage(content=result.content))
    return result.content
print(chat("My name is RZ. I am at BT5153 class"))
print(chat("What is my name? and what class am I in?"))
```

Memory would be “lost”



After a long conversion

Context window fills up fast. Cost more tokens per call.

Memory lost when session ends. -> Need more smart strategy

6. Summary

Prompt Templates

- Prompt Template
  - A pre-defined structure with placeholders for dynamic inputs
  - Example: “Summarize the following text for [audience] in [format]: [text]”
    - Here, [] are placeholders.
  - RAG, Prompt Chaining, Basic memory all belong to prompt template.

The Hierarchy: LLM->App -> Agent

LLM (Raw Model)

Predict Tokens. No interface. No autonomy.

↓

LLM Application

LLM+Prompt Template+Interface. The flow is predefined.

↓

Agent

LLM controls its own flow with memory. Uses tools.

↓

Multi-Agent

Multiple specialized agents collaborate on complex tasks.

Next Class: Agent Fundamentals + Tools